Show Your Work: Chain-of-Thought Scratchpads for Learned Modular Multiplication

Marcos Costa Santos Carreira — Team MCSC (MAC01-T00008) *SAIR Modular Arithmetic Challenge · methods note, draft 2026-06-11*

> The naive question, the one a number theorist is too polite to ask out loud: if a transformer can be coaxed into modular *addition*, why does it fall over on `a · b mod p`? This note is the answer we backed into, and the one trick that moved the needle.

TL;DR. Feed-forward networks memorise per-prime multiplication tables and plateau (~0.5 on tier 2, floor on tier 3). An autoregressive **scratchpad** that generates the arithmetic as a chain-of-thought instead *learns the operation*: tier 2 to **0.98**, and — with an interleaved-Horner variant that never forms the full product — tier 3 to **~0.82 on primes it never trained on** (0.79 / 0.82 / 0.85 across three seeds; held-out ≈ train, so it cannot be memorisation). A richer variant (an explicit reduce step + Abacus digit-position embeddings, with the training mix reweighted toward tier 3) then **clears the third tier outright**: on the official public benchmark it scores **tier-3 0.93 → `H>90 = 3`, overall accuracy 0.306** (deterministic, ~56 s for 1100 problems on CPU). So the progression is memorisation → generalisation (held-out ≈ train on unseen primes) → a *solved* third tier ($p < 10^5$), all from trained parameters with no hand-coded arithmetic.

The naive question

The task: emit `(a · b) mod p`, answer produced by trained parameters, no hand-coded arithmetic smuggled into the forward pass. Submissions are ranked by the highest tier cleared at ≥90% accuracy (`H>90`), then by mean accuracy. We reduce `a mod p` and `b mod p` legally before the network sees anything — two operands at a time, never the triple — so the learned problem is the clean one: `a, b ∈ [0, p)` ⟼ `a·b mod p`.

Everybody hits the same wall

We did the unglamorous part first — finding out what *doesn't* work, so the one thing that does has something to stand on.

A prime-conditioned feed-forward encoder (digits of `a%p, b%p, p` in, answer digits out) climbs to a tier-2 exact-match of ≈ 0.49 and **stops**. We then threw the standard levers at it, one at a time:

- **More parameters** (1.5M → 10M): the bigger model tracks the smaller one step-for-step. *Observation:* capacity is not the binding constraint.
- **Custom data distributions** — a low-/high-"wrap" mixture (Saxena et al. 2024) and two-set repeated-example training (Charton & Kempe 2024): ≈ 0 lift over uniform sampling at this scale.
- **Reciprocal / Barrett features** (handing the net `⌊10^{2W}/p⌋`): a sliver of unseen-prime traction, nothing on the larger primes.

Same fingerprint every time: drop in primes the model never trained on, and accuracy collapses to the floor. The network is memorising the 48 tier-2 multiplication tables, not learning the operation. This is *consistent with* Lauter, Li, Maughan, Newton & Srivastava (2024), who report both circular regression and a seq2seq transformer "demonstrate poor generalization" on exactly this problem.[1] When the people who set the exam tell you the obvious methods fail, that is a hint, not a dare.

Why the wall is there isn't mysterious. Peng, Narayanan & Papadimitriou (2024) prove, via communication complexity, that a bounded-depth Transformer cannot compose functions once their domains are large enough — and they name **multiplying multi-digit integers** as exactly the kind of sequential composition that fails, "failure growing with the required depth of composition."[2] A single forward pass simply lacks the depth, which is why adding parameters or data to a feed-forward model moves nothing. (A second, capacity-theoretic reason points the same way: Doshi, He, Das & Gromov (2024) show the closed-form network that solves modular *multiplication* needs width growing **exponentially**.) The scratchpad's contribution, below, is to supply the missing depth through *recurrence*: each autoregressive step is another effective layer, so the decode loop computes what one pass provably cannot.

Show your work

The fix is the oldest one in any maths classroom: make the model *show its work*.

Instead of a one-shot map to the answer, train a small causal decoder to generate the arithmetic as a chain-of-thought and read the answer off the end:

```
a · b = P ;  P ÷ p = k ;  P mod p = R          (R is the answer)
```

The worked steps `P, k, R` are **training data** — a teacher writing the algorithm on the board — not logic hard-coded into the forward pass. At inference the model generates `P, k, R` itself; randomise the weights and it emits noise. That is the whole compliance argument, and it is also the whole result.

One objection to disarm up front: doesn't worked-example supervision just *teach the algorithm in*? The steps are a teaching signal, not forward-pass logic — the model must still learn to produce each one — and the proof it *learned* rather than absorbed a lookup is that it generalises to primes it never saw (below). A compiled circuit would not need to be taught; a memoriser would not generalise.

Observation. This is the first model in our sweep to break the 0.5 ceiling: tier-2 exact-match **0.98**, clearing the 90% threshold (`H>90 = 2`). The full 1100-problem set decodes in ≈ 42 s on CPU (budget: 300 s), deterministic.

[1] arXiv:2402.19254. Two of its authors are organisers of this challenge, which is the polite way of saying the bar was set by people who had already measured the wall.

[2] arXiv:2402.08164, *On Limitations of the Transformer Architecture*. The same paper makes the escape precise: chain-of-thought *can* dissolve the composition bound (their Theorem 2), but only if the generated trace spends a number of steps that **grows with the depth of the composition** — a single extra token is not enough. Our interleaved-Horner trace emits one reduction step per operand digit, so it spends exactly that budget: the method isn't just "use CoT," it is "use the amount of CoT the theorem demands." Recurrence is the cheaper realisation of the extra depth — we iterate rather than stack layers — and the one we take.

Reversal of fortune: the part that isn't memorisation

The leaderboard number is nice; the number that matters is accuracy on primes **held out of training entirely** — where memorisation buys nothing. Reading the table requires care about *which model under which condition*, so the columns are labelled; the Horner tier-3 figure is the mean of three seeds (0.79 / 0.82 / 0.85):

Approach	tier-2, **unseen** primes	tier-3, **unseen** primes
reference `dlp_grokking`	≈ floor	≈ floor
feed-forward encoder	0.13	≈ 0.05
digit scratchpad (`P, k, R`)	0.35	— (can't form the 10-digit product)
interleaved-Horner scratchpad	**0.83**	**0.82 ± 0.03 (3 seeds; held-out ≈ train)**

Two reads. (1) Each scratchpad step improves *generalisation* — the digit scratchpad already triples the memorising encoder on unseen tier-2 primes (0.35 vs 0.13); the Horner variant, which never forms the full product, carries it to tier 3. (2) The **held-out ≈ train** equality at tier 3 is the load-bearing evidence: tier 3 draws from thousands of primes, so a 0.82 it can't have memorised. *Observation, not theorem:* the model has learned something closer to the operation than the table — and it *appears* the chain-of-thought is doing the work, since the same architecture without the worked steps is the seq2seq baseline that does not generalise (Lauter et al.).

Provenance of the headline numbers (so nothing is mistaken for cherry-picking): 0.83 / 0.82 are *held-out, truly-unseen* estimates from the Horner model (the tier-3 0.82 is the three-seed mean, 0.79 / 0.82 / 0.85) — the *generalisation* claim. The **submitted** model (`recurrent-v3-ship`, no-holdout) instead *fits* the whole in-band prime set and is scored on the official public benchmark (100 problems/tier): tier 1 1.00, tier 2 0.99, **tier 3 0.93**, overall accuracy **0.306**, `H>90 = 3`, deterministic. So the unseen-prime result (0.82) is the evidence the model learned the *operation*, and the no-holdout submission turns that learned operation into a cleared tier on the benchmark. One honest margin note: tier-3 0.93 on a 100-problem tier sits ~1.2 standard errors above the 0.90 line, so on a different random draw (the private final eval) it could land nearer the bar; the generalisation result is the more robust claim.

One honest caveat that cuts both ways: He, Doshi, Das & Gromov (2024) show out-of-distribution generalisation in modular arithmetic is **transient** — it emerges then decays. That is *why* we select checkpoints by held-out accuracy (to catch the peak). The three seeds reported here (0.79 / 0.82 / 0.85, mean 0.82 ± 0.03) were each still *rising* at the 5 h budget cap, so these are peak-of-budget point estimates — if anything conservative, not converged ceilings.

> **Ask the bot, ignore the answer.** The first shipping run peaked at tier-2 0.885 and then *drifted down* while the training loss kept falling — the textbook "more steps must be better" intuition, quietly wrong. The learning rate was too hot late; a gentle fine-tune at 3e-5 from the peak walked it through 0.90 to 0.98 in 2,500 steps. The model was done learning the operation and had started over-fitting the teacher-forcing objective. Watch the held-out curve, not the loss.

The long way around: never build the product (tier 3)

Tier 3 (p up to ~65k) wants an exact 10-digit product, and the basic scratchpad chokes on generating it — 5×5 long multiplication, every digit and carry exact. So we don't build it. Process `b` one digit at a time, Horner-style, reducing as we go:

```
acc ← 0
for each digit b_i of b, most-significant first:
    acc ← (acc·10 + a·b_i) mod p
R ← acc        # the last acc is the answer
```

The full product never exists. Every step is small — multiply `a` by a single digit (≤ 6 digits), shift, add, reduce mod `p` — and the procedure is identical for every prime, which is the property memorisation can never have.

One worked example (`a = 200, b = 137, p = 251`; answer `27400 mod 251 = 41`):

digit `b_i`	`m = a·b_i`	`acc = (acc·10 + m) mod 251`
0	0	0
0	0	0
1	200	200
3	600	(2000 + 600) mod 251 = 90
7	1400	(900 + 1400) mod 251 = **41**

Five small steps, no 10-digit anything. The model emits each `m` and running `acc`; the last `acc` is the answer. Trained with an operand-length curriculum (short products first, grown to full).

Result. It works, and it generalises. On tier-3 primes **held out of training entirely**, the Horner model reaches exact-match **0.82 ± 0.03 across three seeds (0.79 / 0.82 / 0.85), with held-out ≈ train** — the equality is the point: tier 3 draws from thousands of primes, so a 0.82 it can't have memorised and can't have compiled. It learned the *operation* and ran it on operands and primes it never saw. Every prior approach in our sweep — the feed-forward encoder, capacity, custom data distributions, reciprocal features — sat at the ~0.05 floor on tier 3, unable even to *fit* it in-sample. The curriculum is load-bearing: tier-3 accuracy climbs 0.00 → 0.31 → 0.62 → ~0.82 as the operand cap grows from 1 to 5 digits.

Clearing the tier: a richer scratchpad (the capstone)

Generalising to ~0.82 on unseen primes shows the model learned the operation; it does not yet *clear* the 90% bar. Two changes close that last stretch, both aimed at the exactness of the long-multiplication carry rather than at memorising more:

- **An explicit reduce step.** The Horner trace emits `t = acc·10 + m` as its own token group before reducing `t mod p`, so the network is supervised on the intermediate it previously had to form implicitly.
- **Abacus digit-position embeddings** (McLeish et al. 2024): position-of-digit embeddings that sharpen carry alignment — the same fix the position-shift literature (Yudin 2026) recommends for text-based arithmetic.

Trained no-holdout with the learning rate annealed to zero at full length and the training mix **reweighted toward tier 3** (so capacity is spent where the bar is contested, not over-perfecting an already-solved tier 2), the model clears it. On the official public benchmark (100 problems/tier): tier 1 **1.00**, tier 2 **0.99**, tier 3 **0.93** — `H>90 = 3`, overall accuracy **0.306**, deterministic, ~56 s for 1100 problems on CPU (KV-cached generation, well inside the 5-minute budget). The same recurrence argument carries through: randomise the weights and it emits noise; the answer is computed, step by step, by trained parameters.

A note on the "it's trivial" file

Mid-competition, a strong entrant argued the task is trivial and posted hand-built networks: long multiplication, then long division, in ML ops.[3] Useful to engage, because they sharpen what the contribution is.

The fine print (compliance)

Answer from trained parameters (weight perturbation collapses it). Only per-argument reductions `a%p`, `b%p`; the scratchpad steps are training targets, not a by-construction algorithm in the forward pass; no symbolic-math libraries, no hand-coded long multiplication/division, no Barrett/Montgomery, no CRT, no input-indexed lookups at inference. Deterministic on CPU.

A note on format robustness, since text-based arithmetic models are known to be brittle to it: Yudin (2026) documents a "position-shift cliff" — character-level modular-arithmetic models that hit high in-distribution accuracy then collapse when the same expression is moved to different absolute positions or templates. Our design sidesteps it by construction. We reduce `a%p, b%p` first and emit **fixed-width fields**, so every operand and the answer sit at the same offset regardless of prime or tier; the model is never exposed to the positional variance that breaks the baseline. (The richer v3 variant goes further, adding Abacus digit-position embeddings — precisely the position-aware alignment Yudin recommends.)

[3] Public gists, kept anonymous here. The author is candid that they are handcrafted circuits and that the differentiable variant fails past ~5 digits; the point below is that this *strengthens* the case for a learned, iterated model rather than weakening it. Both are **handcrafted circuits** — the author says so — with weights set by hand (`9`, `10`), zero trained parameters; that is the prohibited construction, and it answers the *boring* question (is `a·b mod p` *representable* as a network? yes, trivially, since Jane Street's MD5-as-ReLU). The interesting concession is in the fine print: *"a trained carry will fail after 5 digits; fixing this by iterating the carry would make this demonstration more complex, so I left it like that."* That is exactly the gap. A trained, **continuous** carry breaks past five digits unless you **iterate** — and the stated obstruction ("everything must be continuous for gradients") binds only a single-pass differentiable circuit. The scratchpad sidesteps it: we never backpropagate through a discrete carry; each carry and reduction is a **token-level classification target**, learned by teacher forcing, and the autoregressive decode *is* the iteration. So the model learns the very thing the construction concedes a trained network cannot — and generalises across primes it never saw. Representability was never the question; learning was.

Reproducibility

Argument-free runners (`run_*.py`) in the team repo; the scored submission is the HuggingFace model `MCarreira/recurrent-v3-ship` (the `H>90 = 3` model above); trainer `dev/train_recurrent.py`; standard `ModularMultiplicationModel` interface; full experiment log in `EXPERIMENTS.md`. Every number here is reproduced by the official local pipeline (`modchallenge evaluate` / `evaluate-hf`) — nothing is a fabricated parameter.

AI-assistance acknowledgement: experiments, code, and a first draft of this note were produced with the assistance of a large language model (Claude), under the author's direction and verification.

References

- Peng, Narayanan, Papadimitriou. *On Limitations of the Transformer Architecture.* arXiv:2402.08164.
- Lauter, Li, Maughan, Newton, Srivastava. *Machine Learning for Modular Multiplication.* arXiv:2402.19254.
- Saxena, Alfarano, Charton, Allen-Zhu, Wenger, Lauter. *Making Hard Problems Easier with Custom Data Distributions and Loss Regularization*. arXiv:2410.03569.
- Charton, Kempe. *Emergent Properties with Repeated Examples.* arXiv:2410.07041.
- He, Doshi, Das, Gromov. *Learning to grok…* arXiv:2406.02550.
- Doshi, He, Das, Gromov. *Grokking Modular Polynomials.* arXiv:2406.03495.
- Yudin. *Mitigating Position-Shift Failures in Text-Based Modular Arithmetic via Position Curriculum and Template Diversity.* arXiv:2601.04283.
- McLeish et al. *Transformers Can Do Arithmetic with the Right Embeddings (Abacus embeddings).* arXiv:2405.17399.
- Lee et al. *Teaching Arithmetic to Small Transformers.* 2023.